Optimizing Vision-Based Object Detection and Lane Segmentation for Advanced Driver Assistance Systems Deployed on Resource-Constrained Devices

Mashrafi Mohammad Monon
Computer Science, NYUAD
mashrafi@nyu.edu

Advised by: Muhammad Shafique

ABSTRACT

Advanced Driver Assistance Systems (ADAS) have demonstrated transformative potential in improving vehicular safety by enabling real-time object detection and lane segmentation. However, the widespread adoption of these systems remains limited to high-end vehicles due to the intensive computational demands of deep neural networks (DNNs). This paper presents a two-phase capstone effort aimed at democratizing access to ADAS technologies by optimizing DNN-based perception models for deployment on resource-constrained devices, such as smartphones and embedded systems. In the initial phase, we conducted an extensive evaluation of lightweight object detection and lane segmentation models, identifying candidates with optimal trade-offs between accuracy and computational efficiency. The second phase involved the development of a unified multi-task learning (MTL) architecture with a shared backbone and task-specific heads. We implemented advanced optimization techniques including loss balancing, transfer learning from traffic-specific datasets, and multi-task conflict resolution using PCGrad and Cross-Stitch mechanisms. Empirical results confirm the feasibility of deploying real-time ADAS features on low-power platforms without significantly compromising accuracy. The proposed framework represents a practical step toward more inclusive vehicular safety solutions.

KEYWORDS

ADAS, Object Detection, Lane Segmentation, DNN Optimization, Resource-Constrained Devices, Neural Architecture Search, Pruning

Reference Format:
Mashrafi Mohammad Monon. 2025. Optimizing Vision-Based Object Detection and Lane Segmentation for Advanced Driver Assistance Systems Deployed on Resource-Constrained Devices. In *NYUAD Capstone Project 2 Reports, Spring 2025, Abu Dhabi, UAE.* 9 pages.

1 INTRODUCTION

Advanced Driver Assistance Systems (ADAS) are transforming modern driving by enabling vehicles to autonomously perceive and respond to road environments through features like collision avoidance, object tracking, and lane keeping. These systems rely on deep neural networks (DNNs) to perform real-time object detection and semantic segmentation, tasks that require substantial computational power and energy. As a result, most commercially available ADAS features are currently limited to high-end vehicles equipped with powerful GPUs and dedicated hardware accelerators. This technological constraint creates an equity issue—drivers in low-income regions or owners of mid-range vehicles are often excluded from the safety benefits these systems offer.

The core challenge lies in the gap between the capabilities of state-of-the-art DNNs and the limited processing budgets of edge devices. Smartphones, Raspberry Pi boards, and other embedded platforms typically lack the memory, throughput, and thermal headroom needed to run high-accuracy ADAS models in real time. Bridging this gap requires rethinking how such models are designed, trained, and deployed—emphasizing lightweight architecture, efficient learning strategies, and task-specific adaptation.

To address this need, this capstone project was conducted in two successive phases. In the first phase, we undertook a comprehensive review and benchmarking of lightweight

This report is submitted to NYUAD's capstone repository in fulfillment of NYUAD's Computer Science major graduation requirements.

جامعة نيويورك أبوظبي
NYU ABU DHABI

Capstone Project 2, Spring 2025, Abu Dhabi, UAE

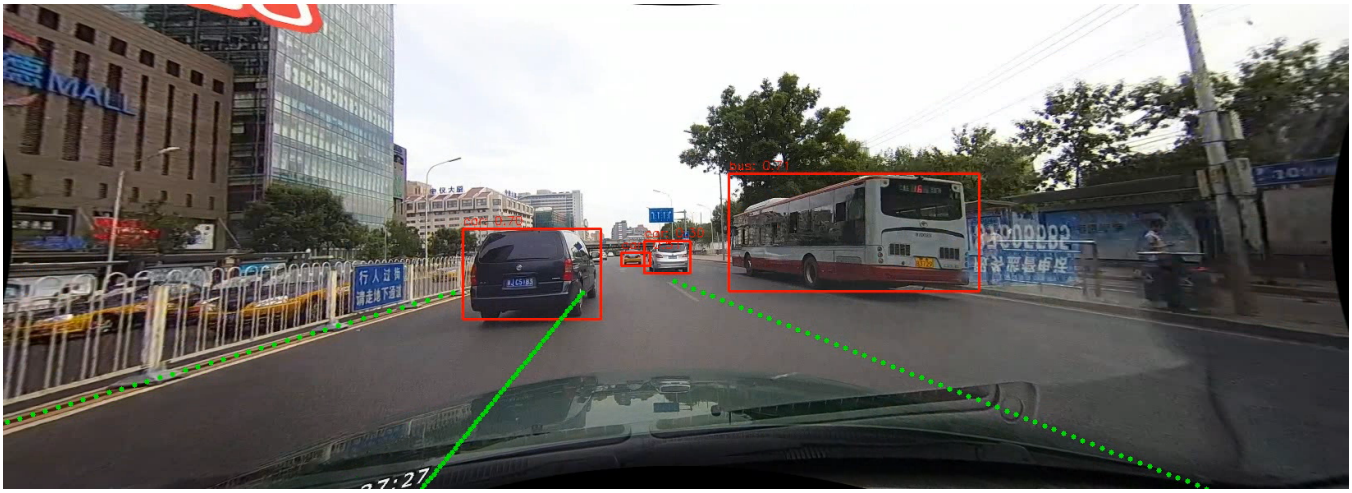

Figure 1: Inference Output from YOLOv8n + CLRNet in Real Driving Scen

models for object detection and lane segmentation. Models were evaluated using metrics such as Average Precision (AP), F1 score, parameter count, and computational cost (FLOPs), with a focus on identifying those suitable for constrained environments. Key insights were gained into the trade-offs between model complexity and task performance, helping to identify optimal candidates such as YOLOv8n for detection and CLRNet for lane segmentation.

In the second phase, we transitioned from evaluation to integration and deployment. A unified multi-task learning (MTL) architecture was developed to execute both tasks concurrently using a shared backbone network. This approach reduces redundancy and leverages common spatial features for greater efficiency. However, combining tasks within a single model introduces new challenges, such as gradient interference, task dominance during training, and mismatches in feature requirements. To overcome these obstacles, we applied a range of strategies including careful initialization from pre-trained models, dataset-specific fine-tuning, weighted loss balancing, and multi-task optimization techniques such as PCGrad and Cross-Stitch networks.

The final system was evaluated on benchmark datasets including CULane and BDD100k, demonstrating the feasibility of deploying a real-time ADAS perception stack on constrained devices without major compromises in accuracy. This work underscores the potential to democratize vehicle safety by extending ADAS capabilities beyond high-end platforms and into more accessible, affordable domains. By tightly coupling lightweight architecture design with multi-task optimization, we present a viable path forward for low-cost, high-impact deployment of intelligent transportation systems.

2 RELATED WORK

Advances in object detection and lane segmentation models have paved the way for their integration into ADAS.

2.1 Object Detection Models

Object detection models can be broadly classified into two categories: two-stage detectors and one-stage detectors.

2.1.1 Two-Stage Detectors. These models, such as Faster R-CNN [9], first generate region proposals and then classify these proposals. They are known for high accuracy, especially in scenarios where precision is critical. However, the computational overhead of running a separate proposal and classification stage makes them unsuitable for real-time ADAS applications on resource-constrained devices.

2.1.2 One-Stage Detectors. These models skip the region proposal step, directly predicting bounding boxes and class probabilities, making them faster and more efficient. Among one-stage detectors:

Transformer-Based Models. Recent innovations like DETR [2] leverage transformer architectures for object detection, offering high performance and scalability. However, their computational demands are still a challenge for resource-constrained environments.

YOLO Family. Renowned for real-time detection capabilities, YOLO models have evolved significantly. Anchor-based versions like YOLOv2 to YOLOv5 rely on predefined bounding boxes for object localization, achieving high precision. Anchor-free versions, starting with YOLOv6 and continuing

to YOLOv8, simplify the architecture, reducing computational costs while maintaining accuracy. Despite these improvements, YOLO models often require further optimization to meet ADAS-specific requirements. [5]

Other Lightweight Object Detection Models. Recent advancements have introduced models specifically designed for resource-constrained environments. PP-PicoDet [15] exemplifies a highly efficient lightweight detector optimized for edge devices, offering competitive performance while maintaining minimal computational requirements. FemtoDet [12] focuses on energy efficiency and employs Instance Boundary Enhancement (IBE) for precise localization, striking a balance between accuracy and computational efficiency. YOLObile [1] integrates pattern-based pruning to reduce model size while maintaining essential network connectivity, enhancing its deployment feasibility for mobile platforms. These models, alongside the YOLO family and transformer-based approaches, represent promising candidates for enabling efficient object detection in ADAS applications.

2.2 Lane Segmentation Models

2.2.1 Pixel-Level Lane Segmentation Models. Models such as U-Net [10] and Deeplabv3 [3] focus on pixel-wise classification, achieving high accuracy in identifying lane boundaries. However, their computational intensity poses challenges for real-time applications in ADAS, especially on resource-constrained devices.

2.2.2 Anchor-Based Lane Detection Models. These models utilize predefined anchors for lane detection and can be further categorized into:

Row Anchor-Based Models. These models use predefined rows to detect lanes, providing a structured approach to lane localization. This method is efficient and suitable for uniform road geometries. UFLD (Ultra-Fast Lane Detection) and its successor UFLDv2 are notable examples. [7, 8] UFLD utilizes row anchors to predict lane positions efficiently, achieving real-time performance with lightweight designs. UFLDv2 builds on this by improving robustness in diverse driving scenarios and enhancing accuracy while retaining speed.

Line Anchor-Based Models. These models leverage predefined lines as anchors, offering better adaptability for roads with varying geometries and more complex lane structures. CLRNet (Cross Layer Refinement Network), LaneATT (Attention-Based Lane Detection), and CondLaneNet (Conditional Lane Detection Network) are prominent examples. CLRNet focuses on refining lane predictions through multi-layer information fusion, while LaneATT utilizes attention mechanisms for precise lane boundary detection. CondLaneNet employs conditional convolution techniques to enhance detection adaptability under varying conditions. [4, 11, 19]

2.3 Multi-Task Learning Models & Optimization Techniques

2.3.1 Unified Multi-Task ADAS Models. Several recent architectures demonstrate how object detection and segmentation tasks can be jointly modeled in real-time. Notably, A-YOLOM [14] builds on the YOLOv8 backbone and introduces a segmentation decoder that outputs both lane and drivable area maps. It uses a combined loss:

$$\mathcal{L}_{\text{seg}} = \lambda_{\text{lane}} \cdot \mathcal{L}_{\text{lane}} + \lambda_{\text{drivable}} \cdot \mathcal{L}_{\text{drivable}}$$

with fixed scalar weights. Detection is handled via standard YOLOv8 losses, and the final loss is the weighted sum of detection and segmentation terms.

YOLOPv3 [17] further emphasizes the segmentation task, assigning greater loss weight to lane detection using a composite of Focal and Tversky losses. This model also integrates self-attention mechanisms in the segmentation head to improve spatial context awareness. Meanwhile, Mobip [18] prioritizes lightweight deployment, using MobileNetV2 as a shared backbone and simplifying task decoders to reduce inference cost. It implicitly balances tasks through architectural simplicity, avoiding complex loss weighting schemes.

HybridNets [13], on the other end of the spectrum, performs detection over all 10 BDD100K classes while also handling two segmentation tasks. It adopts a staged training strategy—first training detection, then segmentation, and finally fine-tuning jointly—which helps mitigate task interference but increases training complexity and time.

These models demonstrate the trade-offs in multi-task design: static scalar balancing offers simplicity, while staged training and attention modules enhance accuracy at increased computational cost. In our work, we adopt a scalar loss balancing approach similar to A-YOLOM, where the total loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + \lambda \cdot \mathcal{L}_{\text{seg}}$$

2.3.2 Gradient Conflict Mitigation: PCGrad and Cross-Stitch. While scalar weighting addresses task dominance at the loss level, it does not resolve conflicts that arise during parameter updates. To this end, gradient-level optimization techniques such as PCGrad [16] and Cross-Stitch [6] Networks offer more principled solutions.

PCGrad (Projected Conflicting Gradient) modifies the gradient of each task to eliminate conflicting components. If the gradient from one task negatively aligns with another ($\mathbf{g}_i^\top \mathbf{g}_j < 0$), PCGrad projects it onto the orthogonal plane of the other's gradient:

$$\mathbf{g}_i \leftarrow \mathbf{g}_i - \frac{\mathbf{g}_i^\top \mathbf{g}_j}{\|\mathbf{g}_j\|^2}\mathbf{g}_j \quad \text{if } \mathbf{g}_i^\top \mathbf{g}_j < 0$$

This ensures task gradients do not hinder each other during shared parameter updates, improving convergence without requiring architectural changes. PCGrad has been shown to accelerate training and improve joint task performance in various MTL settings.

Cross-Stitch Networks, in contrast, learn how to share features across tasks through a linear combination of activations at each layer. Given features $\mathbf{x}_A$ and $\mathbf{x}_B$ from task-specific branches A and B, a cross-stitch unit learns:

$$\begin{bmatrix} \tilde{\mathbf{x}}_A \\ \tilde{\mathbf{x}}_B \end{bmatrix} = \begin{bmatrix} \alpha_{AA} & \alpha_{AB} \\ \alpha_{BA} & \alpha_{BB} \end{bmatrix} \begin{bmatrix} \mathbf{x}_A \\ \mathbf{x}_B \end{bmatrix}$$

are learned during training, enabling the network to adaptively control inter-task sharing. This dynamic mechanism can lead to improved representation learning and task specialization, particularly when tasks differ in complexity or semantic structure.

3 METHODOLOGY

3.1 Overview

The proposed methodology integrates object detection and lane segmentation into a single framework optimized for Advanced Driver Assistance Systems (ADAS). A flowchart is used to visually represent the sequential and interconnected steps of the workflow, offering clarity on how each component contributes to the overall objective. This overview highlights the need for selecting resource-efficient models, combining functionalities seamlessly, and employing optimization techniques to meet real-time performance requirements on constrained devices.

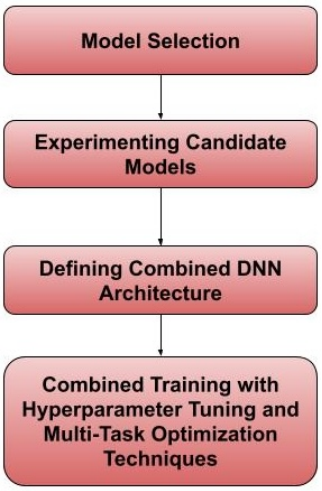

Figure 2: Overview of the Methodology

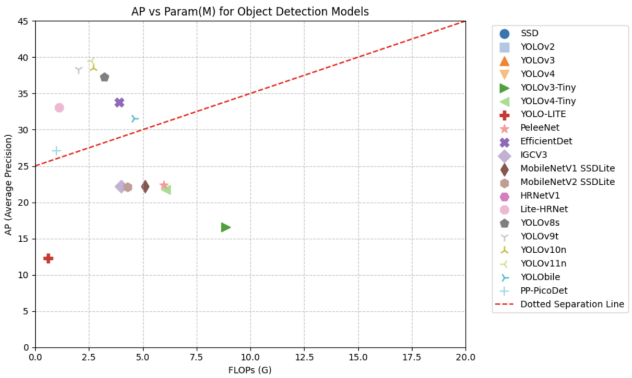

Figure 3: AP vs Params(M) plot on different lightweight Object Detection Models

3.2 Optimum Models Selection

3.2.1 Object Detection. : We evaluate various lightweight object detection models, such as lightweight versions of YOLO models, PP-PicoDet, MobileNet variants, and other efficient object detectors, by analyzing their performance against key metrics such as accuracy, latency, and computational requirements. This evaluation is critical for identifying models that can operate efficiently in resource-constrained environments while meeting the performance demands of ADAS. The two visualizations below play a critical role in guiding model selection:

AP vs Param(M) for Object Detection Models: This plot captures the relationship between Average Precision (AP) and the number of parameters (in millions). By plotting the performance in AP against the parameter count, this visualization allows us to evaluate the trade-off between model complexity and detection performance. This metric is particularly important for identifying models suitable for constrained environments where memory and storage are limited.

AP vs FLOPs for Object Detection Models: This plot examines the relationship between Average Precision (AP) and computational complexity, as measured in FLOPs. By plotting the performance in AP against the required FLOPs, we gain insights into the computational demands of each model and assess their feasibility for real-time deployment on resource-constrained devices. These two visualizations together provide a comprehensive framework for systematically evaluating and comparing object detection models based on their performance and computational efficiency.

After plotting the performance of these models in terms of Average Precision (AP) against the number of parameters and against FLOPs, we identified models positioned above the Pareto-optimal line as the most suitable candidates. Based

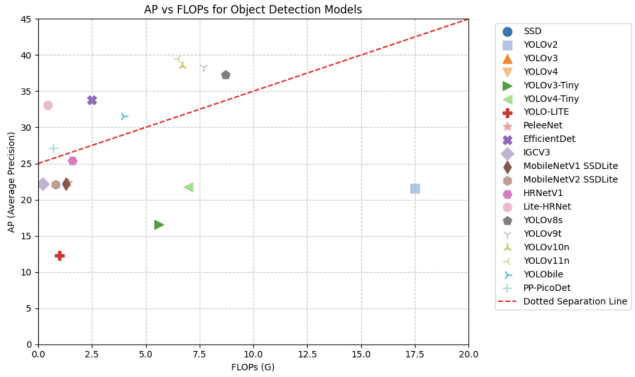

Figure 4: AP vs FLOPs plot on different lightweight Object Detection Models

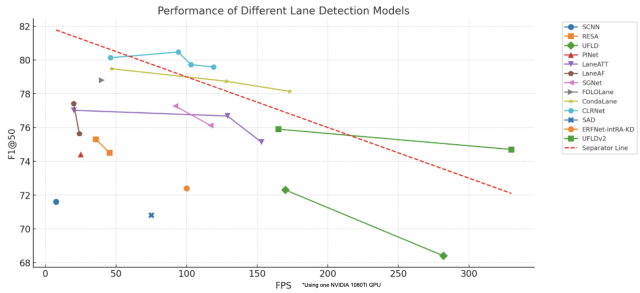

Figure 5: F1 vs FLOPs plot on different Lane Detection Models

on this analysis, we selected YOLOv8n, EfficientDet, LiteHRNet, PP-PicoDet, and YOLObile for their favorable trade-offs between accuracy and computational efficiency. For further experimentation, we are currently focusing on YOLOv8n, given its lightweight design and strong performance relative to our evaluation metrics.

3.2.2 Lane Detection. For lane detection, the evaluation focuses exclusively on anchor-based models such as UFLD, UFLDv2, CLRNet, and LaneATT, which are well-suited for real-time Advanced Driver Assistance Systems (ADAS) applications. These models offer a balance between computational efficiency and detection accuracy, making them ideal for resource-constrained environments. To systematically evaluate the performance of these models, we plot the F1@50 metric against Frames Per Second (FPS). By analyzing the F1@50 vs FPS plot, we can assess the trade-off between detection precision and computational efficiency. This approach enables us to identify models that perform well in terms of both accuracy and real-time capability. Models positioned above the separator line in the plot are considered optimal, as they exhibit superior trade-offs suitable for deployment in resource-constrained ADAS systems.

To identify the most efficient models, we draw a Pareto-optimal line on the F1@50 vs FPS plot. Models positioned on or above this line represent the best trade-offs between accuracy and real-time performance. Based on this analysis, we selected UFLDv2, CLRNet, and CondLaneNet as the optimal candidates for integration into the unified architecture. These models provide robust detection accuracy while maintaining the computational efficiency required for deployment in resource-constrained environments.

3.3 Combined DNN Architecture

The unified architecture was designed to minimize computational redundancy while supporting specialized processing

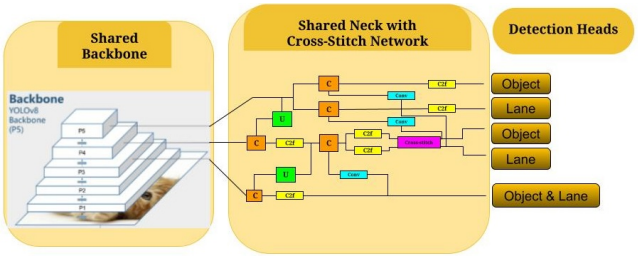

Figure 6: Combined DNN Architecture

for object detection and lane segmentation tasks. To achieve this, we employed YOLOv8n—a compact and efficient object detector—as the shared feature extraction backbone. On top of this backbone, two parallel heads were integrated: the native YOLOv8 object detection head, and a CLRNet-based lane segmentation head.

CLRNet (Cross Layer Refinement Network) was selected for its ability to refine spatial features across multiple stages via progressive upsampling and feature fusion. Unlike lightweight segmentation heads that directly map backbone features to segmentation masks, CLRNet employs a structured decoder with refinement modules, which was crucial for achieving higher accuracy in lane delineation under diverse conditions such as curves, occlusions, and lighting changes.

Between the shared backbone and task-specific heads, we introduced a shared neck module, which aggregates multi-scale features (e.g., from C3 or SPP blocks in YOLOv8) to enable higher-resolution decoding. This shared neck also allowed for experimentation with Cross-Stitch units—lightweight linear blending layers that permit feature exchange between detection and segmentation paths.

By leveraging a single shared backbone, the model significantly reduces memory and compute load compared to two independent models. Figure 6 illustrates the architectural composition.

This architectural design supports end-to-end training, enabling joint optimization of both tasks with shared low-level

features and specialized high-level heads. The use of a shared neck and Cross-Stitch layers further increases flexibility in controlling how much information is shared between tasks, reducing task interference while conserving computation.

3.4 Training Configuration and Scalar Loss Balancing

To ensure efficient and balanced learning across object detection and lane segmentation tasks, a carefully structured training strategy was implemented. The total loss function was defined as a weighted sum of the individual task losses:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{det}} + \lambda \cdot \mathcal{L}_{\text{seg}}$$

Here, $\mathcal{L}_{\text{det}}$ refers to the object detection loss as defined in YOLOv8 (comprising CIoU loss for bounding boxes and binary cross-entropy for class/objectness), while $\mathcal{L}_{\text{seg}}$ includes binary cross-entropy applied to the output of the CLRNet-based lane segmentation head. The scalar λ acts as a task-balancing hyperparameter, modulating the contribution of segmentation to the overall gradient update.

Initial experiments employed equal weighting, but subsequent empirical tuning revealed that segmentation loss tended to dominate due to its dense output nature and smaller gradient scale in detection. A scalar value of $\lambda = 0.1$ provided the most balanced trade-off, allowing detection performance to remain stable while enabling segmentation to improve incrementally. This value was selected as the default for all subsequent experiments.

In addition to scalar tuning, optimizer selection and initialization strategies were explored to further stabilize training. Both `SGD` and `Adam` optimizers were tested. `SGD` provided consistent convergence but plateaued at suboptimal values, while `Adam` delivered faster convergence and higher peak performance, especially for segmentation.

To improve model convergence and performance, we evaluated two initialization strategies: using pretrained weights obtained from training on the `BDD100k` dataset, and using randomly initialized weights. In both cases, the full network was trained end-to-end without freezing any layers. Models initialized with `BDD100k` weights showed significantly faster convergence and more stable segmentation performance, likely due to the transfer of domain-relevant features from the driving dataset.

3.5 Multi-Task Optimization Techniques

To further enhance joint training and reduce adverse interactions between object detection and lane segmentation tasks, we integrated two multi-task optimization techniques: PCGrad and Cross-Stitch Networks. While both are widely studied in multi-task learning literature, their application in resource-constrained ADAS pipelines required careful adaptation.

PCGrad was implemented as an optimizer wrapper around Adam. It required minimal architectural modification, allowing seamless integration into the training loop. During implementation, we observed that PCGrad stabilized early training loss curves, especially when gradients from the two tasks were misaligned. This was particularly beneficial in reducing performance oscillation on the segmentation branch, which typically exhibits noisier gradients than detection.

In contrast, Cross-Stitch units were added at two strategic positions between the shared neck and task-specific heads. These units introduce learnable parameters that control the degree of feature sharing, and are trained end-to-end alongside the rest of the network. While computationally lightweight, Cross-Stitch units increased model flexibility, particularly benefiting lane segmentation performance in challenging scenarios (e.g., shadows, occlusions). Empirically, we found that applying Cross-Stitch only to mid-level features (not deepest layers) yielded the best accuracy-efficiency balance.

To evaluate the effectiveness of these strategies, we trained three configurations: baseline MTL (no gradient adaptation), PCGrad-only, and PCGrad + Cross-Stitch. Figures 9 and 10 show that both methods improved convergence and accuracy, with the combined setup delivering the most stable performance across tasks and scenarios.

4 RESULTS AND DISCUSSION

4.1 Analyzing Backbone Compatibility for Lane Detection

In this section, we evaluate the integration of the YOLOv8n DarkNet backbone with lane detection models, specifically UFLDv2 and CLRNet, to analyze performance differences and explore their feasibility for a unified architecture. The lane detection models were trained on the CULane dataset, while the object detection backbone was pre-trained on different datasets, including COCO and BDD100k.

Initial Experiment with YOLOv8n DarkNet Backbone The YOLOv8n DarkNet backbone was directly integrated with UFLDv2 and CLRNet to evaluate lane detection performance. The results showed that the performance of both UFLDv2 and CLRNet remained mostly consistent, with slight fluctuations, indicating that the backbone could initially support lane detection tasks.

Experiment with Pre-trained DarkNet Backbone (COCO Dataset) We trained the UFLDv2 model using the YOLOv8n backbone pre-trained on the COCO dataset, freezing the backbone during training. The results showed a significant performance drop compared to the benchmark. This suggests that the features learned by the DarkNet backbone on COCO

Figure 7: results on training UFLD with the pre-train backbone frozen

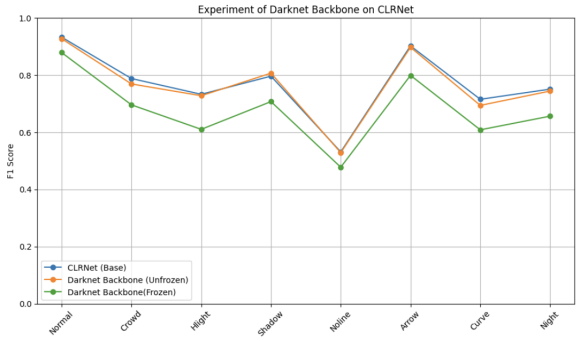

Figure 8: results on training CLRNet with the pre-train backbone frozen

do not generalize well to lane detection tasks, possibly due to a lack of traffic-specific feature representations in the COCO dataset.

Training DarkNet Backbone on Traffic-Specific Dataset (BDD100k) To address the limitation of the COCO dataset, we retrained the YOLOv8n DarkNet backbone on the BDD100k dataset, which focuses on traffic scenarios. When UFLDv2 was trained with this backbone (frozen), the results improved compared to the COCO-trained backbone but still fell short of the benchmark. This finding highlights that traffic-specific training data can enhance feature extraction for lane detection but may not fully resolve the limitations of UFLDv2.

CLRNet Performance on Traffic-Specific DarkNet Backbone The CLRNet model was trained using the BDD100k-pretrained DarkNet backbone (frozen). Interestingly, the performance of CLRNet on this setup was much closer to the benchmark compared to UFLDv2. This suggests that CLRNet's architecture, which includes a Neck module after the backbone, likely contributed to its superior performance by better utilizing the feature representations from the backbone.

Discussion on Model Architectures The disparity in results between UFLDv2 and CLRNet raises important hypotheses. Initially, we assumed that the YOLOv8n backbone might not store sufficient feature representations for lane detection, explaining the poor performance of UFLDv2. However, the strong performance of CLRNet with the same backbone suggests otherwise. A likely explanation is that UFLDv2, which lacks a Neck module, has fewer learnable parameters after the backbone, limiting its ability to utilize extracted features. CLRNet's Neck module may provide the additional learning capacity needed to bridge this gap.

4.2 Unified Training with Loss Scaling and Multi-Task Optimization

This phase of the project focused on developing and evaluating a unified model capable of performing both object detection and lane segmentation using a shared backbone. After selecting YOLOv8n as the detection backbone and CLRNet as the segmentation head, we conducted a series of experiments to identify optimal strategies for joint training. These experiments explored the effects of loss scaling, optimizer selection, gradient conflict resolution, and dynamic feature sharing.

Effect of Loss Scaling The first step involved tuning the task-balancing scalar λ, which adjusts the relative contribution of the segmentation loss to the total loss function. Initial trials with equal weighting (λ=1.0) resulted in poor convergence for the object detection task, as the segmentation gradients dominated training dynamics. Reducing λ to 0.1 provided a better balance, allowing both tasks to improve simultaneously. This value was found to yield stable and consistent training behavior and was used in all subsequent experiments.

Optimizer Comparison: Adam vs SGD To further stabilize training, we experimented with two widely used optimizers: SGD, commonly used in object detection tasks, and Adam, which has shown strong performance in segmentation. SGD produced smoother initial convergence but often plateaued at suboptimal accuracy. Adam, on the other hand, introduced more variation early in training but consistently achieved higher final mAP and F1 scores. Based on this trade-off, Adam was selected as the optimizer for all final runs. While these findings were introduced in the Methodology section, they are reaffirmed here to contextualize performance differences observed later.

Evaluation of PCGrad and Cross-Stitch We then explored two advanced optimization techniques designed to improve multi-task learning:

PCGrad was applied as an optimizer wrapper to mitigate conflicting gradient signals between detection and segmentation tasks. Incorporating PCGrad led to smoother training

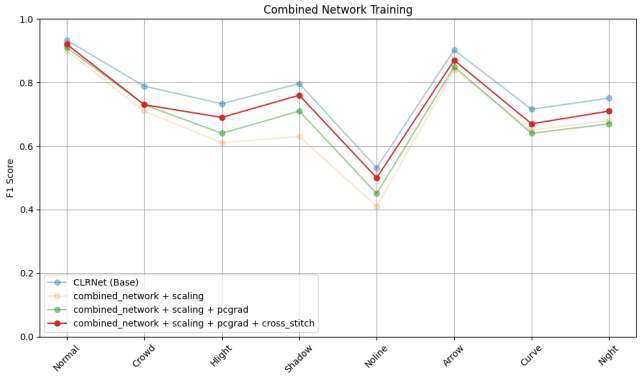

Figure 9: Performance Comparison of Optimization Techniques on Lane Detection

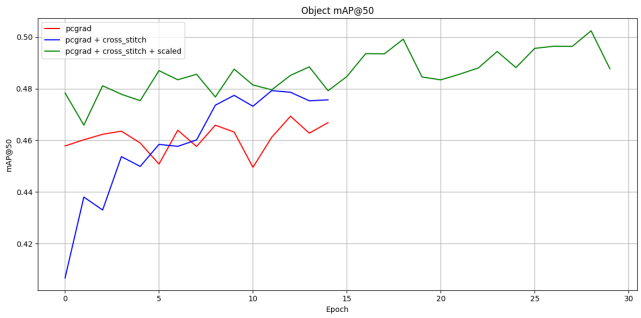

Figure 10: Performance Comparison of Optimization Techniques on Object Detection

curves and marginal improvements in object detection accuracy.

Cross-Stitch Units were integrated into the shared neck to allow feature-level interaction between task-specific branches. These units learned to dynamically share information, leading to improved segmentation outcomes, especially in visually complex conditions like shadows or night scenes.

When combined, PCGrad and Cross-Stitch provided complementary benefits. PCGrad helped maintain task independence during optimization, while Cross-Stitch enabled richer contextual sharing. The resulting model exhibited the best trade-off between detection and segmentation performance.

Final Comparison with Single-Task Baselines To evaluate the overall effectiveness of the unified architecture, we compared our final multi-task model to a combination of two strong single-task baselines: YOLOv8n for object detection and CLRNet for lane segmentation. Each baseline was trained independently to its optimal configuration, without any architectural compromises or multi-task constraints.

Model	Params (M)	Object mAP@50:95	Lane F1@50
YOLOv8n	3.01M	0.34	-
CLRNet	11.8M	-	79.58
Ours	**4.43M**	**0.332**	**73.4**

Figure 11: Final Result Compared to Baselines

As summarized in the results table, the unified model performs slightly below the combined baselines in both detection and segmentation metrics. This gap is most noticeable in lane detection. However, it's important to note that the standalone CLRNet baseline uses a larger backbone and a deeper Neck module, giving it more capacity to specialize in lane features—something the unified model, by design, cannot fully replicate due to parameter sharing and efficiency constraints.

Despite this, our unified model achieves competitive accuracy, staying within 1–2% of the baseline scores, while using just 4.43M parameters compared to 14.81M required when deploying both YOLOv8n and full CLRNet separately. This represents a 70% reduction in model size, making the unified architecture far more efficient for real-time applications.

The result highlights a clear trade-off: while some performance is sacrificed, the gains in compactness, inference simplicity, and deployment feasibility are substantial. For resource-constrained platforms such as embedded ADAS systems, this trade-off is not only acceptable—it is often necessary.

5 CONCLUSION

This study presented a unified perception architecture for Advanced Driver Assistance Systems (ADAS), integrating object detection and lane segmentation into a single model optimized for efficiency. Through systematic exploration of model architectures and training strategies, we demonstrated that both tasks can be effectively learned together without significant compromises in performance.

The results confirmed that with the right backbone design and optimization techniques, a multi-task model can achieve competitive accuracy while substantially reducing computational complexity. This unified approach offers a practical and scalable solution for deploying real-time perception systems on resource-limited platforms.

REFERENCES

[1] Yuxuan Cai, Hongjia Li, Geng Yuan, Wei Niu, Yanyu Li, Xulong Tang, Bin Ren, and Yanzhi Wang. 2020. YOLObile: Real-Time Object Detection on Mobile Devices via Compression-Compilation Co-Design. *ArXiv* abs/2009.05697 (2020). https://api.semanticscholar.org/CorpusID:221655546

[2] Nicolas Carion, Francisco Massa, Gabriel Synnaeve, Nicolas Usunier, Alexander Kirillov, and Sergey Zagoruyko. 2020. End-to-End Object Detection with Transformers. arXiv:cs.CV/2005.12872 https://arxiv.org/abs/2005.12872

[3] Liang-Chieh Chen, George Papandreou, Florian Schroff, and Hartwig Adam. 2017. Rethinking Atrous Convolution for Semantic Image Segmentation. arXiv:cs.CV/1706.05587 https://arxiv.org/abs/1706.05587

[4] Lizhe Liu, Xiaohao Chen, Siyu Zhu, and Ping Tan. 2023. CondLaneNet: a Top-to-down Lane Detection Framework Based on Conditional Convolution. arXiv:cs.CV/2105.05003 https://arxiv.org/abs/2105.05003

[5] Zonglei Lyu, Tong Yu, Fuxi Pan, Yilin Zhang, Jia Luo, Dan Zhang, Yiren Chen, Bo Zhang, and Guangyao Li. 2023. A survey of model compression strategies for object detection. *Multimedia Tools and Applications* (Nov. 2023). https://doi.org/10.1007/s11042-023-17192-x

[6] Ishan Misra, Abhinav Shrivastava, Abhinav Gupta, and Martial Hebert. 2016. Cross-Stitch Networks for Multi-task Learning. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 3994–4003.

[7] Zequn Qin, Huanyu Wang, and Xi Li. 2020. Ultra Fast Structure-aware Deep Lane Detection. arXiv:cs.CV/2004.11757 https://arxiv.org/abs/2004.11757

[8] Zequn Qin, Pengyi Zhang, and Xi Li. 2022. Ultra Fast Deep Lane Detection with Hybrid Anchor Driven Ordinal Classification. arXiv:cs.CV/2206.07389 https://arxiv.org/abs/2206.07389

[9] Shaoqing Ren, Kaiming He, Ross Girshick, and Jian Sun. 2016. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. arXiv:cs.CV/1506.01497 https://arxiv.org/abs/1506.01497

[10] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-Net: Convolutional Networks for Biomedical Image Segmentation. arXiv:cs.CV/1505.04597 https://arxiv.org/abs/1505.04597

[11] Lucas Tabelini, Rodrigo Berriel, Thiago M. Paixão, Claudine Badue, Alberto F. De Souza, and Thiago Oliveira-Santos. 2020. Keep your Eyes on the Lane: Real-time Attention-guided Lane Detection. arXiv:cs.CV/2010.12035 https://arxiv.org/abs/2010.12035

[12] Peng Tu, Xu Xie, Guo Ai, Yuexiang Li, Yawen Huang, and Yefeng Zheng. 2023. FemtoDet: An Object Detection Baseline for Energy Versus Performance Tradeoffs. In *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. 13272–13281. https://doi.org/10.1109/ICCV51070.2023.01225

[13] Dat Vu, Bao Ngo, and Hung Phan. 2022. HybridNets: End-to-End Perception Network. *arXiv preprint arXiv:2203.09035* (2022).

[14] Jiayuan Wang, Q. M. Jonathan Wu, and Ning Zhang. 2024. You Only Look at Once for Real-Time and Generic Multi-Task. *IEEE Transactions on Vehicular Technology* 73, 9 (2024), 12625–12637. https://doi.org/10.1109/TVT.2024.3394350

[15] Guanghua Yu, Qinyao Chang, Wenyu Lv, Chang Xu, Cheng Cui, Wei Ji, Qingqing Dang, Kaipeng Deng, Guanzhong Wang, Yuning Du, Baohua Lai, Qiwen Liu, Xiaoguang Hu, Dianhai Yu, and Yanjun Ma. 2021. PP-PicoDet: A Better Real-Time Object Detector on Mobile Devices. arXiv:cs.CV/2111.00902

[16] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, and Karol Hausman. 2020. Gradient Surgery for Multi-Task Learning. In *Advances in Neural Information Processing Systems*, Vol. 33. 5824–5836.

[17] Y. Zhan, H. Liu, and Y. Wang. 2024. YOLOPv3: Multi-Task Visual Perception for Object Detection and Semantic Segmentation in Intelligent Driving. *Remote Sensing* 16, 10 (2024), 1774. https://doi.org/10.3390/rs16101774

[18] Y. Zhang, X. Li, and J. Wang. 2023. Mobip: A Lightweight Model for Driving Perception Using Multi-Task Learning. *Frontiers in Neurorobotics* 17 (2023), 1291875. https://doi.org/10.3389/fnbot.2023.1291875

[19] Tu Zheng, Yifei Huang, Yang Liu, Wenjian Tang, Zheng Yang, Deng Cai, and Xiaofei He. 2022. CLRNet: Cross Layer Refinement Network for Lane Detection. arXiv:cs.CV/2203.10350 https://arxiv.org/abs/2203.10350